Technical Analysis Report

Waste Image Classification — Computer Vision Pipeline

Course: Introduction to Computer Vision

Institute: Epicode Institute of Technology

Author: Giovanni Iannuario s00014624

Date: May 2026

---

1. Problem Statement

1.1 Motivation

Improper waste disposal represents one of the most critical environmental challenges of our time. According to the World Bank, global solid waste generation is expected to reach 3.4 billion tonnes per year by 2050. A significant fraction of recyclable materials — cardboard, glass, metal, paper, plastic — ends up in landfills due to incorrect sorting, either by consumers or by manual operators at sorting facilities. Manual waste classification is slow, inconsistent, and physically demanding.

Automated image-based waste classification can address this gap by providing fast, consistent, and scalable sorting decisions. A camera-based system integrated into a conveyor belt or recycling bin could pre-sort waste streams with minimal human intervention, reducing contamination and improving recycling rates.

1.2 Task Definition

This project frames waste sorting as a **closed-set, multi-class image classification** problem. Given an input image, the system must assign it to one of six material categories: **cardboard, glass, metal, paper, plastic,** and **trash**. The last category acts as a catch-all for items that do not belong to any recyclable class.

The primary evaluation metric is **macro F1-score**, chosen because it assigns equal weight to all classes regardless of frequency. This is critical given the severe class imbalance in the available data: the `trash` category contains only 137 images compared to roughly 500 for other classes. Accuracy alone would mask poor performance on minority classes.

1.3 Dataset: TrashNet

The dataset used is **TrashNet** (Thung & Yang, 2016), a widely cited benchmark for waste classification research. It contains 2,527 images of individual waste items photographed against white backgrounds under controlled lighting conditions. Images were sourced via Kaggle (`feyzazkefe/trashnet`), an alternative mirror of the original dataset.

Class distribution:

Class	Images	% of Total
cardboard	403	15.9%
glass	501	19.8%
metal	410	16.2%
paper	594	23.5%
plastic	482	19.1%
trash	137	5.4%
Total	**2527**	**100%**

The `trash` class is severely underrepresented at 5.4% of the total. This imbalance poses a direct challenge to both training and evaluation, and is discussed further in Section 4.

---

2. Methodology

2.1 Data Acquisition and Preprocessing Pipeline

All images are loaded via `torchvision.datasets.ImageFolder`, which automatically assigns class labels from directory structure. The full dataset is split into **train (70%) / validation (15%) / test (15%)** using a fixed random seed (42), yielding 1,768 / 379 / 380 samples respectively. A stratified split is achieved by fixing the generator seed before `random_split`.

Validation and test preprocessing:

1. Resize to 224×224 pixels (bilinear interpolation)
2. Convert to tensor (pixel values in [0, 1])
3. Normalize with ImageNet statistics: μ = [0.485, 0.456, 0.406], σ = [0.229, 0.224, 0.225]

Training augmentation pipeline:

1. Resize to 256×256 (slightly larger than target)
2. RandomCrop(224) — random spatial offset provides translation invariance
3. RandomHorizontalFlip (p=0.5)
4. RandomVerticalFlip (p=0.5) — waste items can appear in any orientation
5. RandomRotation(±30°) — handles non-canonical orientations
6. ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3, hue=0.1) — simulates lighting variation
7. ToTensor + ImageNet normalization

Augmentation is applied **only at training time** via a dedicated `AugDataset` wrapper class that loads images directly from disk, bypassing the base dataset's cached val-mode transform. This ensures no augmented images are used during evaluation (no data leakage).

Note on num_workers: Python 3.14 changed the Windows multiprocessing start method to `spawn`, which requires all DataLoader worker arguments to be picklable. The `AugDataset` wrapper is defined at module level (not as a local class) to satisfy this constraint. `num_workers` is set to 0 to ensure compatibility.

2.2 Classical Baseline: HOG + SVM

Feature Extraction — HOG

Histogram of Oriented Gradients (HOG) is a hand-crafted descriptor that encodes the local distribution of gradient directions in an image (Dalal & Triggs, 2005). The algorithm:

1. Computes image gradients (Sobel filters in x and y directions)
2. Divides the image into small cells (8×8 pixels)
3. Computes a histogram of gradient orientations (9 bins, covering 0–180°) within each cell
4. Groups cells into overlapping blocks (2×2 cells) and L2-normalizes each block descriptor
5. Concatenates all block descriptors into a single feature vector

For a 224×224 RGB image with these parameters, the output is a **31,752-dimensional vector**. HOG captures edge and texture information that is relevant for distinguishing material categories: corrugated cardboard produces regular diagonal gradients, glass produces sparse edge patterns, metal can produce specular highlights and smooth regions.

HOG parameters used:

- orientations = 9
- pixels_per_cell = (8, 8)
- cells_per_block = (2, 2)
- channel_axis = -1 (compute HOG on each RGB channel separately)

Classifier — SVM with RBF Kernel

Features are standardized with `sklearn.preprocessing.StandardScaler` (zero mean, unit variance per dimension) before being passed to a Support Vector Machine with Radial Basis Function kernel. The full pipeline is wrapped in an `sklearn.pipeline.Pipeline` for clean cross-validation compatibility.

Hyperparameters: C = 10, γ = 0.001 (set based on common practice for high-dimensional HOG features; a grid search was explored with `--grid_search` flag).

The SVM finds a maximum-margin decision boundary in the kernel-induced feature space. With RBF kernel, the effective dimensionality of the feature space is infinite, providing sufficient expressive power for non-linearly separable material classes.

Training time: approximately 15 minutes on CPU (dominant cost: HOG extraction for 1,768 training images + SVM kernel matrix computation).

2.3 Deep Learning: EfficientNet-B0 Fine-Tuning

Architecture

EfficientNet-B0 (Tan & Le, 2019) is a compound-scaled convolutional neural network pretrained on ImageNet-1k. It achieves state-of-the-art accuracy/efficiency trade-offs by jointly scaling depth, width, and resolution using a fixed compound coefficient. Key properties:

- **Parameters:** ~5.3M (backbone) — suitable for CPU training
- **Input resolution:** 224×224
- **Feature dimension at global average pool:** 1,280

The original classification head (Linear(1280, 1000)) is replaced with a custom head:

```
Dropout(p=0.3)
Linear(1280, 256)
ReLU
Linear(256, 6)
```

Dropout(0.3) regularizes the head and reduces overfitting given the small dataset size. The intermediate layer (256 neurons) provides a compact learned representation before the final 6-class projection.

Two-Phase Transfer Learning

Transfer learning from ImageNet is applied in two phases to avoid catastrophically overwriting pretrained features before the new head has converged:

Phase 1 — Head training (5 epochs, frozen backbone):

- All backbone parameters are frozen (`requires_grad = False`)
- Only the custom head is trained
- Optimizer: Adam, lr = 1e-3
- Loss: CrossEntropyLoss

Phase 2 — Full fine-tuning (up to 25 epochs):

- All parameters unfrozen
- Optimizer: Adam, lr = 1e-4 (lower rate to avoid overwriting pretrained features)
- LR Scheduler: ReduceLROnPlateau (factor=0.3, patience=3) — reduces learning rate on validation loss plateau
- Early stopping with patience = 7 on validation loss
- Best model checkpoint saved to `models/efficientnet_best.pth`

Rationale for transfer learning: ImageNet-pretrained features capture low-level textures, edges, and color patterns that are directly relevant to material classification. The fine-tuning approach has been shown to outperform training from scratch on datasets of this size (~2,500 images).

2.4 Grad-CAM Visual Explanations

Gradient-weighted Class Activation Mapping (Selvaraju et al., 2017) provides human-interpretable visual explanations of the model's predictions. For a given input and predicted class c:

1. Forward pass through the network; record the final convolutional feature map A (shape: H×W×K, K channels)
2. Compute the gradient of class score y^c with respect to A
3. Global average pool the gradient over spatial dimensions to obtain channel weights α_k
4. Compute the weighted combination: CAM = ReLU($\Sigma_k \alpha_k \cdot A_k$)
5. Upsample the CAM to input image resolution (224×224)
6. Normalize to [0, 1] and apply JET colormap for visualization

The ReLU ensures only features with positive influence on the predicted class are highlighted. The resulting heatmap is blended with the original image (55% original, 45% heatmap) in the Gradio application.

2.5 Gradio Web Application

The interactive inference interface (`app.py`) is built with Gradio Blocks and offers two modes:

Analyze tab: accepts image upload, webcam snapshot, or clipboard paste. Produces:

- Grad-CAM overlay heatmap
- Per-class confidence horizontal bar chart (matplotlib)
- Top-3 prediction summary with icons

Real-time tab: streams the webcam feed and runs inference on each frame. Produces:

- Annotated video frame with predicted class and confidence bar drawn with OpenCV
- Top-3 prediction summary updated live

The real-time mode omits Grad-CAM to maintain interactive frame rates (~2–4 FPS on CPU).

---

3. Experimental Results

3.1 Training Curves — EfficientNet-B0

Phase	Epoch	Train Loss	Train Acc	Val Loss	Val Acc
P1	1	1.271	60.0%	1.006	74.4%
P1	3	0.976	74.6%	0.918	77.8%
P1	5	0.915	77.3%	0.887	80.5%
P2	2	0.784	84.0%	0.756	86.5%
P2	5	0.639	91.0%	0.665	90.2%
P2	9	0.554	95.9%	0.627	92.6%

Phase	Epoch	Train Loss	Train Acc	Val Loss	Val Acc
P2	12	0.524	97.4%	0.609	93.9%
P2	20	0.503	98.0%	0.589	93.7%
P2	25	0.491	98.7%	0.599	93.4%

The model converged smoothly across both phases. Phase 1 brought validation accuracy from random (~17%) to 80.5% in 5 epochs by training only the head. Phase 2 improved further to a peak of 93.9% at epoch 12. The best checkpoint (epoch 20, val_loss=0.5893) was saved for evaluation. Training ran for the full 25 epochs without triggering early stopping.

3.2 HOG + SVM Results

Split	Accuracy	Macro Precision	Macro Recall	Macro F1
Val	65.7%	0.710	0.606	0.626
Test	63.7%	0.700	0.597	0.620

Per-class Test Results:

Class	Precision	Recall	F1-Score	Support
cardboard	0.82	0.67	0.74	61
glass	0.53	0.63	0.58	75
metal	0.52	0.53	0.53	62
paper	0.71	0.84	0.77	89
plastic	0.58	0.53	0.55	72
trash	1.00	0.38	0.55	21

3.3 EfficientNet-B0 Results

Split	Accuracy	Macro Precision	Macro Recall	Macro F1
Val	93.7%	—	—	—
Test	**93.2%**	**0.932**	**0.924**	**0.924**

Per-class Test Results:

Class	Precision	Recall	F1-Score	Support
cardboard	1.00	0.92	0.96	59
glass	0.95	0.96	0.96	78
metal	0.86	0.98	0.92	61
paper	0.95	0.96	0.95	92

Class	Precision	Recall	F1-Score	Support
plastic	0.93	0.85	0.89	66
trash	0.88	0.88	0.88	24

3.4 Model Comparison

Model	Test Accuracy	Macro F1	Parameters	Training Time
HOG + SVM	63.7%	0.620	N/A	~15 min (CPU)
EfficientNet-B0	**93.2%**	**0.924**	~5.3M	~30 min (CPU)

The deep model outperforms the classical baseline by **+29.5 percentage points** in accuracy and **+0.304** in macro F1.

3.5 Per-class Analysis

Best performing classes (EfficientNet-B0):

- **Cardboard** (F1=0.96): Distinctive corrugated texture and brown color are learned reliably. Perfect precision (1.00) — no false positives.
- **Glass** (F1=0.96): Transparent/reflective properties create unique visual signatures that EfficientNet captures well.
- **Paper** (F1=0.95): Flat, matte texture with consistent whitish coloring.

Most improved over SVM:

- **Metal** (SVM F1=0.53 → CNN F1=0.92): The largest gain (+0.39). Metal's specular reflections and smooth surfaces are poorly captured by HOG gradients, but EfficientNet's learned color and texture features handle them well. Recall improved from 0.53 to 0.98.
- **Trash** (SVM F1=0.55 → CNN F1=0.88): Despite having only 24 test samples, the deep model achieves 0.88 F1, demonstrating stronger generalization from limited examples via pretrained representations.

Remaining weakness:

- **Plastic** (F1=0.89): The lowest-performing class for EfficientNet. Plastic items vary greatly in shape, color, and transparency, making them the most visually heterogeneous category.

---

4. Failure Analysis

4.1 HOG + SVM Failure Modes

Metal-Plastic confusion: Both classes frequently contain smooth, featureless surfaces with reflective properties. HOG edge histograms are dominated by the image borders rather than material-specific texture when the surface lacks detail. The confusion matrix shows significant cross-predictions between these two classes.

Trash recall (0.38): With only 21 test samples, the SVM has seen very few `trash` training examples (~96 images at 70% split). The model defaults to assigning ambiguous items to other classes. Perfect precision (1.00) confirms that when `trash` is predicted, it is correct — but the majority of `trash` items are missed.

Global descriptor limitations: HOG operates over fixed-size cells and produces a single global descriptor per image. It does not capture spatial hierarchy or semantic content. A trash can, a metal can, and a plastic bottle may produce similar HOG responses if their local gradient distributions are similar.

4.2 EfficientNet-B0 Failure Modes

Plastic misclassification: The 15% error rate on plastic items is primarily due to the extreme visual diversity of the class. Plastic bags, bottles, containers, and film have very different appearances. Some plastic items (clear bottles) are visually similar to glass.

Real-world distribution shift: All TrashNet images are photographed on white backgrounds under controlled lighting. When the Gradio app is tested with real webcam footage (objects held in hand, complex background), the model's Grad-CAM heatmaps show attention on background regions rather than the object itself. Performance in real-world deployment is expected to be significantly lower than the 93.2% reported here.

Confidence calibration: For highly confident predictions (>90%), the model is generally correct. However, in the 40-70% confidence range, predictions are often wrong. The model does not output well-calibrated probabilities — a temperature scaling post-processing step could improve this.

4.3 Limitations of the Dataset

- **Controlled conditions only:** No images from outdoor settings, bins, or conveyor belts
- **Single objects per image:** Real waste often involves multiple items or partially occluded objects
- **No soiling or deformation:** Real recyclables are often dirty, crushed, or damaged
- **Western bias:** Product types and materials reflect specific consumer markets

---

5. Ethical Considerations

5.1 Environmental Impact of Classification Errors

In a waste sorting application, not all errors are equal. Misclassifying a recyclable item as `trash` (false negative for recyclables) sends recoverable material to landfill, contributing directly to environmental damage. Conversely, contaminating a recycling stream with non-recyclables reduces the quality of the recovered material and may render entire batches unrecyclable.

Recommendation: In deployment, the classification threshold for recyclable classes should be lowered to favour recall over precision — accepting more false positives (recyclables sent for

manual verification) over false negatives (recyclables sent to landfill). A cost-sensitive learning approach (weighted cross-entropy) could be explored in future work.

5.2 Dataset Bias and Representativeness

TrashNet was collected by students at Stanford University under laboratory conditions. Several biases affect the generalizability of models trained on it:

- **Lighting bias:** Controlled white-background photography does not represent real sorting environments
- **Geographic bias:** Products photographed reflect North American consumer goods; performance may degrade on waste from different markets
- **Category completeness:** The six categories do not cover all waste types (e.g., electronic waste, organic waste, textiles)

Any deployment of this model must be preceded by validation on data collected in the target environment.

5.3 Class Imbalance and the Trash Category

The `trash` class (5.4% of images) is a heterogeneous catch-all. Items that do not fit clean material categories are aggregated here, creating an inherently inconsistent class. A model that underperforms on `trash` may systematically misroute atypical items. In practice, a high-recall `trash` detector should be supplemented with manual inspection of uncertain predictions.

5.4 Transparency and Explainability

Grad-CAM visualizations are provided to give operators insight into model decisions. However, Grad-CAM has known limitations: heatmaps are approximate, can highlight spurious correlations (e.g., background color), and may not reflect the true computational path. They should be used as a debugging tool rather than as a definitive explanation.

The application is open source and fully reproducible from the provided code and publicly available dataset.

5.5 Privacy

The TrashNet dataset contains only images of objects and no personally identifiable information. The Gradio application processes images locally; no user images are transmitted to external services, stored, or logged. The real-time webcam mode processes frames entirely on-device.

---

6. Conclusions

This project implemented and evaluated a complete Computer Vision pipeline for waste image classification:

• A **HOG + SVM baseline** achieved 63.7% test accuracy (Macro F1 = 0.620), demonstrating that hand-crafted features provide a reasonable starting point but are limited by their inability to capture material-specific semantic features.

• An **EfficientNet-B0** fine-tuned with two-phase transfer learning achieved 93.2% test accuracy (Macro F1 = 0.924), a substantial improvement of +29.5pp driven by learned hierarchical representations. All six classes exceed F1 = 0.88.

• A **Gradio web application** provides interactive inference with Grad-CAM explanations and real-time webcam classification, satisfying the bonus requirement for web app deployment.

The main limitation is the controlled-condition nature of TrashNet. Future work should address real-world generalization through domain adaptation, data collection in target environments, and cost-sensitive training to prioritize recall on recyclable classes.

---

7. References

1. Thung, G., & Yang, M. (2016). *Classification of trash for recyclability status.* CS229 Project Report, Stanford University.

2. Tan, M., & Le, Q. V. (2019). *EfficientNet: Rethinking model scaling for convolutional neural networks.* ICML 2019. arXiv:1905.11946.

3. Dalal, N., & Triggs, B. (2005). *Histograms of oriented gradients for human detection.* CVPR 2005, pp. 886–893.

4. Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). *Grad-CAM: Visual explanations from deep networks via gradient-based localization.* ICCV 2017, pp. 618–626.

5. Cortes, C., & Vapnik, V. (1995). *Support-vector networks.* Machine Learning, 20(3), 273–297.

6. He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep residual learning for image recognition.* CVPR 2016.

7. Chollet, F. (2017). *Xception: Deep learning with depthwise separable convolutions.* CVPR 2017.

8. World Bank (2018). *What a Waste 2.0: A Global Snapshot of Solid Waste Management to 2050.* Washington, DC.